



Tecnológico de Monterrey

Avance de proyecto 6

Equipo “Financial Sentinel”

Isabella Montiel Reyes - A01278286

Iker Rodríguez - A01712814

Axel Orlando Arenas Vergara - A01278654

Construcción de software y toma de decisiones (Gpo 501)

Profesores:

Enrique Alfonso Calderón Balderas

Denisse Lizbeth Maldonado wFlores

Alejandro Fernández Vilchis

15/05/2026

1. Introducción

El presente documento describe el avance actual del proyecto Financiamiento Sentinela, una plataforma orientada al monitoreo de operaciones financieras, gestión de clientes y administración de alertas para una SOFOM. El sistema se encuentra desarrollado con una arquitectura cliente-servidor utilizando HTML, CSS y JavaScript para el frontend, mientras que el backend utiliza Node.js con Express y PostgreSQL.

El avance contempla:

- 90% del análisis de requisitos funcionales.
- 30% del diseño e implementación.
- Diseño y ejecución de pruebas.
- Diagrama de despliegue.
- Decisiones y seguimiento del proyecto.

2. Arquitectura General del Proyecto

Tecnologías utilizadas

Frontend

- HTML5
- CSS3
- JavaScript

Backend

- Node.js
- Express.js
- JWT para autenticación
- bcryptjs para cifrado

Base de Datos

- PostgreSQL

Librerías principales

- cors
- dotenv
- pg
- jsonwebtoken

3. Avance del 90% del análisis de requisitos funcionales

3.1 Módulo de autenticación

RF-01 Inicio de sesión

Descripción: El sistema debe permitir que los usuarios inicien sesión utilizando correo y contraseña.

Entradas:

- Correo
- Contraseña

Proceso:

- Validación de credenciales.
- Generación de token JWT.

Salidas:

- Token de autenticación.
- Acceso al dashboard.

Estado: Implementado.

3.2 Módulo Dashboard

RF-02 Visualización de métricas

Descripción: El sistema debe mostrar métricas generales del sistema financiero.

Indicadores:

- Total de clientes.
- Total de alertas.
- Total de operaciones.
- Operaciones recientes.

Estado: Implementado parcialmente.

3.3 Módulo de clientes**RF-03 Registro de clientes**

Descripción: El sistema debe permitir registrar clientes nuevos.

Datos requeridos:

- RFC
- CURP
- Nombre completo
- Fecha de nacimiento
- Nacionalidad
- País de nacimiento
- Género
- Estado civil
- Teléfono celular
- Teléfono fijo
- Correo electrónico
- Indicador PEP
- Indicador de cuenta propia

Reglas de negocio:

- RFC único.
- CURP única.
- Correo válido.

Estado: Implementado parcialmente.

RF-04 Consulta de clientes

Descripción: El sistema debe permitir visualizar clientes registrados.

Funciones:

- Consultar tabla de clientes.
- Visualizar información general.

Estado: Implementado.

3.4 Módulo de operaciones

RF-05 Registro de operaciones

Descripción: El sistema debe registrar depósitos y retiros.

Entradas:

- Contrato
- Monto
- Tipo de movimiento

Reglas:

- El monto debe ser positivo.
- El tipo debe ser Depósito o Retiro.

Estado: Implementado parcialmente.

RF-06 Consulta de operaciones recientes

Descripción: El sistema debe mostrar operaciones recientes.

Estado: Implementado.

3.5 Módulo de alertas

RF-07 Visualización de alertas

Descripción: El sistema debe mostrar alertas generadas por operaciones sospechosas.

Datos mostrados:

- Cliente
- Regla activada
- Estatus
- Fecha

Estado: Implementado.

RF-08 Cambio de estatus de alertas

Descripción: El usuario podrá dictaminar alertas.

Estatus permitidos:

- Nueva
- Investigando
- Falsa alarma
- Reportada a CNBV

Estado: Implementado parcialmente.

3.6 Módulo de contratos**RF-09 Consulta de contratos**

Descripción: El sistema debe permitir consultar contratos asociados a un cliente.

Estado: Implementado.

4. Avance del 30% del diseño e implementación**4.1 Arquitectura implementada**

El proyecto actualmente utiliza una arquitectura multicapa:

- Capa de presentación (Frontend).
- Capa lógica (Backend Express).
- Capa de persistencia (PostgreSQL).

La estructura del backend se encuentra organizada mediante:

- Controllers
- Routes
- Models
- Middleware
- DB Connection

4.2 Estructura identificada del proyecto

```
C:\
├── README.md
├── backend
│   ├── .env
│   ├── .gitignore
│   ├── package-lock.json
│   ├── package.json
│   └── server.js
├── controllers
│   ├── alertaController.js
│   ├── authController.js
│   ├── clienteController.js
│   ├── contratoController.js
│   ├── dashboardController.js
│   └── operacionController.js
├── db
│   └── connection.js
├── middleware
│   └── authMiddleware.js
├── models
│   ├── alertaModel.js
│   ├── authModel.js
│   ├── clientModel.js
│   ├── contratoModel.js
│   ├── dashboardModel.js
│   ├── operacionModel.js
│   └── perfilModel.js
└── routes
    ├── alertaRoutes.js
    ├── authRoutes.js
    ├── clientRoutes.js
    ├── contratoRoutes.js
    ├── dashboardRoutes.js
    └── operacionRoutes.js
```

```
├── frontend
│   ├── alertas.html
│   ├── clientes.html
│   ├── dashboard.html
│   └── login.html
├── css
│   ├── components.css
│   ├── layout.css
│   └── variables.css
├── js
│   ├── api
│   │   ├── alertasApi.js
│   │   ├── apiClient.js
│   │   ├── authApi.js
│   │   ├── clientesApi.js
│   │   ├── contratosApi.js
│   │   ├── dashboardApi.js
│   │   └── operacionesApi.js
│   ├── components
│   │   ├── modales.js
│   │   └── sidebar.js
│   └── views
│       ├── alertas.js
│       ├── clients.js
│       ├── dashboard.js
│       └── login.js
```

4.3 Funcionalidades implementadas

Backend implementado

Funcionalidad	Estado
Login con JWT	Implementado
Middleware de autenticación	Implementado
Consulta de dashboard	Implementado
CRUD básico de clientes	Parcial
Registro de operaciones	Parcial
Consulta de alertas	Implementado
Cambio de estatus de alertas	Parcial
Consulta de contratos	Implementado

4.4 Base de datos

El sistema utiliza PostgreSQL como gestor de base de datos.

Tablas identificadas

- usuarios
- clientes
- perfiles_transaccionales
- operaciones
- alertas
- contratos

Relaciones principales

- Un cliente puede tener múltiples contratos.
- Un contrato puede tener múltiples operaciones.
- Una operación puede generar alertas.
- Un usuario puede dictaminar alertas.

4.5 Scripts de base de datos sugeridos

Para la implementación de la base de datos del sistema Sentinel se desarrollaron scripts SQL en PostgreSQL. Estos scripts permiten crear la estructura principal del sistema y registrar datos de prueba para validar el funcionamiento del backend y frontend.

El archivo estructura.sql contiene la creación de tablas, secuencias, relaciones, funciones y restricciones necesarias para el sistema. Entre las tablas principales se encuentran:

- * clientes
- * contratos
- * operaciones
- * alertas
- * productos
- * reglas_monitoreo
- * perfiles_transaccionales
- * reportes_internos

Además, se implementaron funciones y mecanismos de auditoría para registrar cambios importantes en perfiles transaccionales y reglas de monitoreo.

El archivo datos.sql contiene registros de prueba utilizados para validar las consultas y el funcionamiento general del sistema. Estos datos incluyen clientes, contratos, productos financieros y operaciones.

A continuación se muestra un fragmento de los scripts utilizados:

```
CREATE TABLE public.alertas (  
    id integer NOT NULL,  
    cliente_id integer NOT NULL,
```

```

    operacion_id integer,
    regla_id integer NOT NULL,
    estatus character varying(50) DEFAULT 'Nueva',
    fecha_generacion timestamp DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO clientes (
    rfc,
    curp,
    nombre_completo,
    fecha_nacimiento,
    nacionalidad
)
VALUES (
    'GOCJ800101AAA',
    'GOCJ800101HDFXXX01',
    'Juan Manuel González de Cossío',
    '1980-01-01',
    'Mexicana'
);

```

5. Diseño y ejecución de pruebas

5.1 Estrategia utilizada

Para el diseño de pruebas se utilizó:

- Casos de uso.
- Clases de equivalencia.
- Valores frontera.
- Escenarios alternos.

Las pruebas se enfocaron en:

- Autenticación.
- Registro de clientes.
- Registro de operaciones.
- Consulta de alertas.

5.2 Casos de prueba

ID	Caso de prueba	Entrada	Resultado esperado	Estado
TC-01	Login correcto	Correo y contraseña válidos	Acceso autorizado y token JWT	Exitoso
TC-02	Login incorrecto	Contraseña inválida	Error de autenticación	Exitoso
TC-03	Registrar cliente válido	Datos completos	Cliente registrado	Exitoso
TC-04	Registrar cliente con RFC duplicado	RFC repetido	Error de validación	Exitoso
TC-05	Registrar operación válida	Depósito de 5000	Operación registrada	Exitoso
TC-06	Registrar operación negativa	Monto -100	Error de validación	Exitoso
TC-07	Consultar alertas	GET alertas	Lista de alertas	Exitoso
TC-08	Cambiar estatus alerta	Estatus Investigando	Actualización correcta	Parcial

5.3 Pruebas de equivalencia

Registro de operaciones

Clase	Entrada	Resultado esperado
Inválida	monto < 0	Error
Válida	monto > 0	Registro exitoso
Frontera	monto = 0	Validar restricción

5.4 Evidencia de ejecución

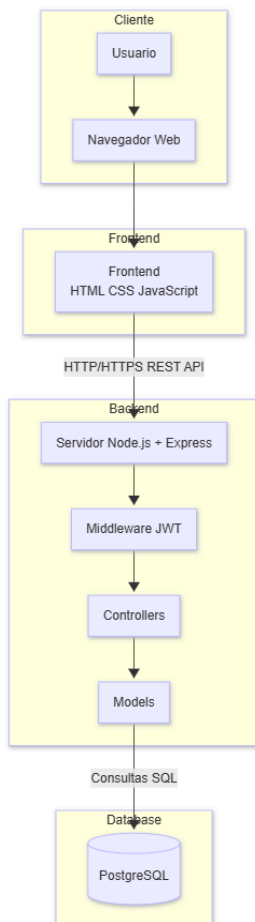
Pruebas realizadas

- Validación de login mediante Postman.
- Validación de endpoints protegidos.
- Consulta de dashboard.
- Registro de clientes.
- Registro de operaciones.

Correcciones realizadas

- Ajuste de validaciones de token JWT.
- Corrección de rutas protegidas.
- Corrección de respuestas JSON.
- Manejo de errores en operaciones.

6. Diagrama de despliegue



Descripción textual del despliegue

El sistema Financiamiento Sentinel opera bajo una arquitectura cliente-servidor. Los usuarios acceden al sistema mediante un navegador web desde una computadora personal o dispositivo móvil. El frontend fue desarrollado utilizando HTML, CSS y JavaScript, y se encarga de mostrar la interfaz gráfica y realizar solicitudes HTTP al backend.

El backend se encuentra implementado con Node.js y Express. Este componente procesa las peticiones del frontend, ejecuta la lógica de negocio, valida la autenticación mediante JWT y administra las operaciones relacionadas con clientes, contratos, operaciones y alertas.

La información del sistema se almacena en una base de datos PostgreSQL, la cual mantiene persistencia de usuarios, clientes, operaciones financieras y alertas generadas.

La comunicación entre frontend y backend se realiza mediante peticiones REST utilizando HTTP/HTTPS. El backend se conecta directamente con PostgreSQL mediante consultas SQL y modelos de acceso a datos.

7. Decisiones y avance del proyecto

7.1 Qué se comprometieron a hacer

- Implementar autenticación segura.
- Crear módulos de clientes y operaciones.
- Integrar dashboard financiero.
- Diseñar pruebas funcionales.
- Implementar conexión a PostgreSQL.

7.2 Qué sí lograron

- Implementación de backend funcional.
- Implementación de autenticación JWT.
- Implementación de rutas protegidas.
- Desarrollo de frontend inicial.
- Integración con PostgreSQL.
- Implementación parcial de dashboard.
- Desarrollo de pruebas funcionales iniciales.

7.3 Qué no lograron y por qué

Actividad	Motivo
Completar validaciones avanzadas	Tiempo limitado
Implementar reportes completos	Prioridad a backend
Automatización completa de pruebas	Curva de aprendizaje
Despliegue en nube	Proyecto aún en desarrollo

7.4 Qué harán en el siguiente avance

- Completar CRUD de clientes.
- Completar módulo de reportes.
- Implementar pruebas automatizadas.
- Mejorar validaciones.
- Desplegar sistema en servidor.
- Implementar bitácora de auditoría.

7.5 Responsables de actividades

Actividad	Responsable
Backend Node.js	Iker
Frontend HTML/CSS/JS	Axel
Base de datos PostgreSQL	Iker
Pruebas y documentación	Isabella
Integración y despliegue	Isabella

8. Conclusión

El proyecto Finacial Sentinel presenta un avance sólido en la implementación de los requisitos funcionales principales. Actualmente se cuenta con una arquitectura funcional cliente-servidor, autenticación segura mediante JWT, conexión con PostgreSQL y módulos esenciales para la operación del sistema.

El siguiente avance se enfocará principalmente en completar funcionalidades pendientes, fortalecer pruebas automatizadas y preparar el sistema para un entorno de despliegue más cercano a producción.